

Waradly — Testing

How to test Waradly: what exists today, how to run things locally, what to automate first, and the manual checklists to run before every deploy.

Last updated	30 September 2026
Related docs	TRD.md · APP_FLOW.md · IMPLEMENTATION_PLAN.md

1. Current state

What	Status
Automated tests for the live backend (functions/)	None. <code>npm test</code> in <code>functions/</code> runs <code>node --test</code> , but there are no test files yet.
<code>tests/*.test.ts</code> in the repo root (5 files, Vitest)	Written for the retired Next.js code in <code>src/lib/</code> . They do not test what runs in production. Treat them as a checklist of what to port (§3.2).
Website and mobile app	Manual testing only (§5).
Deploy check	<code>firebase deploy --only functions --dry-run</code> confirms the backend loads and packages.

Until §3 is done, the manual checklists in §5 and the smoke test in §7 are the safety net.

2. Environments

2.1 Local — Firebase Emulator Suite

Runs everything on your machine with an empty database. Nothing touches production.

```
cd C:\Users\zaina\Projects\waradly\functions
npm install
cd ..
firebase emulators:start
```

Emulator	Port
Hosting (website)	http://localhost:5000
Functions	5001
Firestore	8080

Emulator	Port
Auth	9099
Storage	9199
Emulator UI (browse data, users, logs)	shown in the terminal

Notes:

- Requires Java (for the Firestore emulator).
- With no `RESEND_API_KEY`, emails are **printed to the functions log** instead of sent. Auth emails (verify/reset) appear in the **Auth emulator** tab of the Emulator UI — copy the link from there.
- To make a local admin: register any user, then in the Emulator UI set `role` to `admin` on its `users` document.

2.2 Production

`https://waradly-1.web.app`. Test there only with clearly named test accounts (e.g. `yourname+buyer1@gmail.com`) and delete test data afterwards. Never test payments with real cards before Paymob's sandbox has passed.

3. Automated tests

3.1 Setup

Use Node's built-in test runner — no extra dependencies. Put files in `functions/test/` named `*.test.js`, then:

```
cd C:\Users\zaina\Projects\waradly\functions
npm test
```

3.2 What to automate first (pure logic, no database)

Priority	Module	What to assert
1	<code>stateMachines/rfq.js</code> , <code>offer.js</code> , <code>order.js</code>	Every allowed transition returns <code>true</code> ; forbidden ones (e.g. <code>CLOSED → DRAFT</code> , <code>COMPLETED → PRODUCTION</code>) return <code>false</code> ; editable statuses are correct
1	<code>validation/auth.js</code>	Password rules (8+ chars, letter, number); phone format; <code>terms_accepted</code> must be <code>true</code> ; email lower-cased
1	<code>services/messageDetectService.js</code>	Emails, URLs and phone numbers are replaced with <code>[redacted]</code> and <code>flagged</code> is <code>true</code> ; clean text is untouched
2	<code>masking/*.js</code>	Supplier view of an RFQ never includes <code>rejection_reason</code> / <code>admin_notes</code> ; buyer never sees supplier identity
2	<code>config/fileConstraints.js</code>	Magic-byte check accepts real PNG/JPEG/PDF headers and rejects a renamed file; filename sanitising

Priority	Module	What to assert
2	validation/offer.js	Positive price and MOQ required; optional fields bounded to 1,000 chars
3	Subscription (Phase 3)	Paymob HMAC: correct signature passes, one changed field fails; <code>assertCanSubmitOffer</code> allows the first offer, blocks the second without a subscription, allows it with one; webhook is idempotent

These map to the old files: `stateMachines.test.ts`, `validation.test.ts`, `contactDetection.test.ts`, `masking.test.ts`, `security.test.ts`.

3.3 Example — `functions/test/core.test.js`

```
const test = require('node:test');
const assert = require('node:assert/strict');
const { canTransitionRfq, isRfqEditable } = require('../stateMachines/rfq');
const { canTransitionOrder } = require('../stateMachines/order');
const { scanMessage } = require('../services/messageDetectService');
const { registerSchema } = require('../validation/auth');

test('RFQ state machine', () => {
  assert.equal(canTransitionRfq('DRAFT', 'SUBMITTED'), true);
  assert.equal(canTransitionRfq('RECEIVING_OFFERS', 'EXPIRED'), true);
  assert.equal(canTransitionRfq('CLOSED', 'DRAFT'), false);
  assert.equal(isRfqEditable('PUBLISHED'), false);
});

test('order cannot leave COMPLETED', () => {
  assert.equal(canTransitionOrder('COMPLETED', 'PRODUCTION'), false);
  assert.equal(canTransitionOrder('SAMPLE_REVIEW', 'DISPUTED'), true);
});

test('contact details in messages are redacted', () => {
  const r = scanMessage('Call me on +20 100 123 4567 or mail a@b.com');
  assert.equal(r.flagged, true);
  assert.doesNotMatch(r.maskedContent, /a@b\.com|4567/);
  assert.equal(scanMessage('Can you do 5,000 boxes?').flagged, false);
});

test('registration rejects weak passwords and missing terms', () => {
  const base = {
    email: 'Buyer@Example.com', phone: '+201001234567', username: 'buyer', password: 'Waradly2026',
    role: 'buyer', legal_name: 'Acme', country: 'Egypt', general_region: 'Cairo', terms_accepted: true,
  };
  assert.equal(registerSchema.safeParse(base).success, true);
  assert.equal(registerSchema.safeParse({ ...base, password: 'abcd' }).success, false);
  assert.equal(registerSchema.safeParse({ ...base, terms_accepted: false }).success, false);
});
```

3.4 API tests against the emulators (next step)

Start the emulators (`firebase emulators:exec --only functions,firestore,auth,storage "npm --prefix functions test"`) and call `http://127.0.0.1:5001/waradly-1/us-central1/api/api/...` from tests to cover:

- register → login blocked until verified → verify → login;
- 5 wrong passwords → 429 RATE_LIMITED ;
- buyer cannot read another buyer's RFQ (404); supplier cannot call /api/admin/* (403);
- full happy path: RFQ → approve → offer → accept → order statuses → rate;
- second offer on the same RFQ → 409 ; offer after deadline → 409 .

4. Test accounts

Role	Suggested email	How to set up
Buyer	you+buyer@gmail.com	Register as Buyer, verify email
Supplier	you+supplier@gmail.com	Register as Supplier, verify, complete identity check, apply for a category; approve as admin
Admin	you+admin@gmail.com	Register, then set role: "admin" on its users document in Firestore

Gmail's + addresses all arrive in your normal inbox. Passwords must be 8+ characters with a letter and a number.

5. Manual test checklists

5.1 Public site and accounts

- ☐ Landing page loads; header shows **Log in / Sign up** top right; sections animate in on scroll
- ☐ "Join as a supplier" opens register with Supplier selected
- ☐ Register with a weak password → clear field message, no generic "Validation failed."
- ☐ Register successfully → verification email arrives (check spam) → link verifies
- ☐ Log in before verifying → blocked with "Please verify your email..."
- ☐ 5 wrong passwords → locked for 15 minutes
- ☐ Forgot password → email arrives → reset works → old sessions logged out
- ☐ Logged-in user visiting /login is sent to their dashboard

5.2 Buyer

- ☐ Dashboard shows correct counts; empty states when there's no data
- ☐ Create RFQ as draft, edit, attach a PDF, submit
- ☐ Cannot edit after publishing
- ☐ Rejected RFQ shows the reason; edit and resubmit works
- ☐ Compare offers shows anonymised suppliers; accept one → order created, other offers no longer acceptable
- ☐ Approve and reject a sample; rejecting opens a dispute
- ☐ Rate a completed order; reorder creates a new RFQ/order
- ☐ Profile: change username, upload avatar

5.3 Supplier

- ☐ Identity check and document upload complete
- ☐ Apply for a category; RFQ feed empty until approved, then shows matching RFQs only
- ☐ Submit an offer; a second offer on the same RFQ is refused
- ☐ Edit before deadline; withdraw
- ☐ Order actions: start production → submit sample → mark completed
- ☐ Performance page shows anonymised ID, completed orders, average rating
- ☐ (Phase 3) Second offer on another RFQ without a subscription → Subscribe prompt; after paying → allowed

5.4 Admin

- ☐ Dashboard counts match the queues
- ☐ Approve / reject supplier verification (open original documents)
- ☐ Approve / reject RFQ; adjust distribution
- ☐ Confirm payment and move an order through every status; evidence required at hub, QC and delivery
- ☐ Resolve a dispute both ways (reopen production / cancel)
- ☐ Suspend a user → their next request fails immediately; reactivate
- ☐ Message with an email/phone is masked and appears in Message Flags
- ☐ Audit log records each action above

5.5 Messaging and identity protection

- ☐ Buyer and supplier never see each other's company name, email or phone anywhere (pages, files, messages)
- ☐ Downloaded shared documents are watermarked

6. Cross-device, accessibility and email

Check	How
Layouts	Chrome DevTools device toolbar at 390 px (phone), 768 px (tablet), 1366 px and 1920 px (desktop)
No sideways scrolling on phone	Every page at 390 px
Browsers	Chrome, Edge, Firefox, Safari (iPhone)
Reduced motion	OS setting "reduce motion" on → no animations, all content visible
Keyboard	Tab through login and register; focus ring visible on inputs
Emails	Arrive in Gmail and Outlook; branded HTML renders; button and link work (in inbox, not spam)
Caching	After deploy, pages pick up new CSS/JS on a normal reload

7. Deploy checklist

Before deploying

- ☐ `npm test` passes in `functions/` (once tests exist)
- ☐ `firebase deploy --only functions --dry-run` succeeds
- ☐ Changed pages checked in the local emulator at phone and desktop widths
- ☐ No secrets added to tracked files (`git diff` — keys belong only in `functions/.env.waradly-1`)

Deploy

```
cd C:\Users\zaina\Projects\waradly
firebase deploy --only "functions,hosting"
```

If it fails with "Timeout after 10000", run it again; if it repeats, set `$env:FUNCTIONS_DISCOVERY_TIMEOUT=60` first.

Smoke test after deploying (5 minutes, on waradly-1.web.app)

- ☐ `https://waradly-1.web.app/api/health` responds
- ☐ Landing page, login and register load; logo and favicon show
- ☐ Log in as buyer, supplier and admin — each dashboard loads data
- ☐ `sitemap.xml` and `robots.txt` load
- ☐ `firebase functions:list` shows runtime `nodejs22`

8. Security testing

Test	Expected
Call any <code>/api/admin/*</code> with a buyer token	403
Request another company's RFQ / order / file by ID	404
Upload a <code>.exe</code> renamed to <code>.pdf</code>	Rejected (magic-byte check)
Upload a file > 10 MB	Rejected
Open a file link after 5 minutes	Expired
Read Firestore or Storage directly with the web API key	Denied by rules
Tampered Paymob webhook (<i>Phase 3</i>)	Rejected, subscription unchanged
Search the repo and git history for keys	None found (last checked 30 Sep 2026)

9. Reporting a bug

```
Title:      <what broke, one line>
Where:     <page URL or API endpoint>
Account/role: <buyer / supplier / admin, test email>
Steps:     1. ... 2. ... 3. ...
Expected:  <what should happen>
Actual:    <what happened, with the exact error message>
Evidence:  <screenshot; Functions log line from Firebase console → Functions → Logs>
Device:    <browser + phone/desktop>
```